声视融合异常设备定位系统

摘　要

针对便携式设备巡检中异响定位与现场排查的需求，本作品基于ZYNQ-7020 SoC 平台，设计并实现了一套集多通道声源可视化定位、实时视觉辅助观测与机械臂联动于一体的声视融合检测系统。系统采用“PL 实时计算 + PS Linux 上层控制”的异构协同架构，前端集成 16 路 ICS-43434 数字麦克风双同心圆阵列与 OV5640 摄像头。

在硬件逻辑（PL）侧，系统对 16 路 46.875 kHz 同步音频流执行 200 Hz 高通预处理与 64×36 网格延时求和（Delay-and-Sum）波束成形计算，生成声场伪彩热图；视频链路基于 AXI4-Stream、VDMA 与 Video Mixer 构建流水线，支持原图、灰度、二值化等六种图像模式与矩形 ROI 局部处理，实现 1280×720 摄像头画面、声学热图与 DDR 双缓冲 RGBA 界面的分层合成。在处理器（PS）侧，基于 PetaLinux 构建后台服务，完成背景噪声门控、静音滞回、动态色阶平滑和位置稳定判定；同时支持 2048 点单通道 FFT 主频估计、声源跟随 ROI、原图与测量数据关联保存，以及带任务号和状态回传的机械臂区域控制。

实现报告显示，设计满足所建立的时序约束；声学测试验证了高通与门控策略对显示稳定性的改善，五组合成单音的离线频率估计最大绝对偏差为 0.307 Hz。系统完成了触摸交互、图像模式控制、抓拍相册及机械臂区域任务联调，为设备异响辅助定位和现场记录提供了可运行的平台。

目　录

摘　要 1
第一部分　作品概述 1
1.1 功能与特性 1
1.2 应用领域 2
1.3 主要技术特点 2
1.4 主要性能指标 3
1.5 主要创新点 3
1.6 设计流程 3
第二部分　系统组成及功能说明 4
2.1 整体介绍 4
2.1.1 设计目标与项目基础 4
2.1.2 系统总体架构 4
2.1.3 软硬件划分与接口关系 5
2.1.4 系统工作流程 6
2.2 硬件系统介绍 7
2.2.1 硬件整体介绍 7
2.2.2 机械布局与阵列结构 7
2.2.3 电路与接口模块 8
2.2.4 PL 逻辑与数据通路 10
2.2.5 控制接口、时钟与跨域设计 12
2.3 软件系统介绍 13
2.3.1 软件整体介绍 13
2.3.2 软件各模块介绍 14
2.4 大模型协作与技能包提炼 17
2.4.1 协作目标与任务划分 17
2.4.2 问题定位与假设修正 17
2.4.3 对照验证与结果 18
2.4.4 技能提炼与适用边界 18

2.5 工程复现与运行方法 19
2.5.1 最小复现准备 19
2.5.2 SD 卡写入 20
2.5.3 上电启动 20
2.5.4 首次校准 21
2.5.5 功能验收 21
第三部分 完成情况及性能参数 21
3.1 整体介绍 21
3.2 工程成果 22
3.2.1 机械成果 22
3.2.2 电路成果 23
3.2.3 软件成果 23
3.3 特性成果 24
3.3.1 声源可视化定位与主峰频率显示 24
3.3.2 背景抑制与热点稳定性改善 25
3.3.3 图像处理与触摸交互成果 25
3.3.4 原图抓拍与测量记录成果 26
3.3.5 机械臂区域任务与串口通信成果 27
3.3.6 FPGA 资源与时序成果 27
第四部分 总结 28
4.1 可扩展之处 28
4.2 心得体会 28
第五部分 参考文献 30
附录 A 复现与协作资料 31
A.1 工程入口与构建运行 31
A.2 大模型协作摘录与技能复用 33
A.3 指标说明 33

第一部分　作品概述

1.1 功能与特性

本作品面向便携式巡检，基于 ZYNQ-7020 构建集声源定位、视觉辅助观测与机械臂联动于一体的声视融合检测系统。前端集成 16 路 MEMS 麦克风双环阵列与 OV5640 摄像头，通过延时求和算法生成声场热图，并与实时视频叠加显示，直观呈现声源位置。

系统采用 PL 实时处理、Linux 上层控制的协同架构，支持声学识别逻辑开关、单通道主频估计，以及原图、灰度、二值化、亮度偏移、反相和增益六种图像模式，可选择全屏、手动框选或声源跟随区域处理。触摸界面显示实时坐标、定位稳定状态、芯片温度及机械臂状态，支持摄像头原图与配套测量信息保存、相册浏览和参数调整。系统通过串口提供机械臂夹取、声源跟踪及测试动作接口，支持手动与自动控制，为异常点检查与自主复检功能扩展提供基础。



图 1-1 整机实物及运行界面

1.2 应用领域

本作品主要面向近距离、可听频段的声视辅助检测，可应用于工业设备巡检、生产现场排查及教学科研等场景。

在工业巡检中，系统可对风机、电机等设备的运行声音进行可视化呈现，辅助人员寻找异响区域，并结合实时图像、局部区域处理和抓拍记录开展现场排查。在生产检测中，可用于观察产品运行时的发声位置及声音变化，为人工判断和检测记录提供依据。

在机器人协作场景中，系统可将声源定位结果转换为区域信息，通过串口与机械臂联动，为目标夹取、声源跟踪及后续复检提供控制接口。在教学与科研中，可作为麦克风阵列定位、FPGA 并行处理、嵌入式 Linux 和多模态交互的综合实验平台。

1.3 主要技术特点

（1）PS/PL 协同处理架构。基于 ZYNQ-7020，将声学计算与视频处理部署于 PL，Linux 负责状态管理、触摸交互及外设控制，通过 AXI 与 UIO 寄存器接口实现软硬件协同。

（2）多通道声源定位与稳定显示。采用 16 路麦克风同步采样，结合 200 Hz 高通预处理，在 64×36 网格上完成延时求和计算；配合静音门限、滞回、动态色阶及时间平滑，改善背景闪色与热点波动，并通过位置稳定判定筛选自动控制目标。

（3）视频与界面分层合成。利用 AXI4-Stream、VDMA 和 DDR 组织视频数据流，实现图像处理、声学热图及 RGBA 界面合成；Linux 渲染界面并采用双缓冲刷新，支持独立收放面板与触摸操作。

（4）测量记录与外设联动。通过单通道 FFT 估计主频，配套保存摄像头原图与测量信息；机械臂串口采用任务号及结果回传机制，支持命令关联与执行状态管理。

1.4 主要性能指标

表 1-1 主要性能指标

指标	配置值 / 实测值
麦克风通道数	16 路
声学采样率	46.875 kHz
声学计算网格	64×36
视频分辨率	1280×720
现场观察频段	800～5000 Hz
定位引导距离	约 5 m（粗测）
主频显示对照偏差	约±2 Hz（手机单音）
LUT / FF / BRAM / DSP	48.13% / 43.67% / 92.14% / 60.45%
WNS / WHS	+0.136 / +0.036 ns

1.5 主要创新点

（1）**声源引导的区域处理：**将声学定位结果用于图像 ROI 跟随，实现声源位置与局部图像处理联动。

（2）**声学显示与控制稳定化：**结合高通预处理、静音滞回、动态色阶及位置稳定判定，改善背景闪色与热点波动，筛选稳定控制目标。

（3）检测与执行协同：将有效声源坐标映射为机械臂操作区域，手动模式按最新实时位置确认，自动模式使用稳定目标，通过任务号和结果回传管理执行状态。

1.6 设计流程

先分析便携巡检需求，确定 PS/PL 功能划分与接口；完成阵列适配、高通预处理和视频链路设计，部署 Linux、触摸界面、抓拍相册及机械臂通信。通过仿真、ILA 与板端对照验证采样和控制，优化显示稳定性，开展性能与资源测试，整理复现资料和技能包。

第二部分　系统组成及功能说明

2.1 整体介绍

2.1.1 设计目标与项目基础

本作品面向便携式设备巡检中的异响定位、现场观察与执行联动需求，设计了一套基于 ZYNQ-7020 的声视融合检测系统。系统集成麦克风阵列、摄像头、HDMI 触摸屏及机械臂通信接口，将声音的空间分布与实时图像结合，为使用者提供声源位置、图像处理、测量记录和外设控制功能。

系统以开源声学相机工程为基础，继承其麦克风采集、延时求和定位及视频显示链路。在此基础上，针对实际使用的双环 16 麦阵列完成几何参数适配，并增加高通预处理、声学显示稳定化、Linux 上层控制、触摸交互、图像模式与 ROI 处理、抓拍相册及机械臂串口联动，形成便于现场操作和后续扩展的系统。[1]

2.1.2 系统总体架构

系统由感知前端、PL 实时处理、PS 上层控制、人机交互及机械臂执行五部分组成，总体结构如图 2-1 所示。

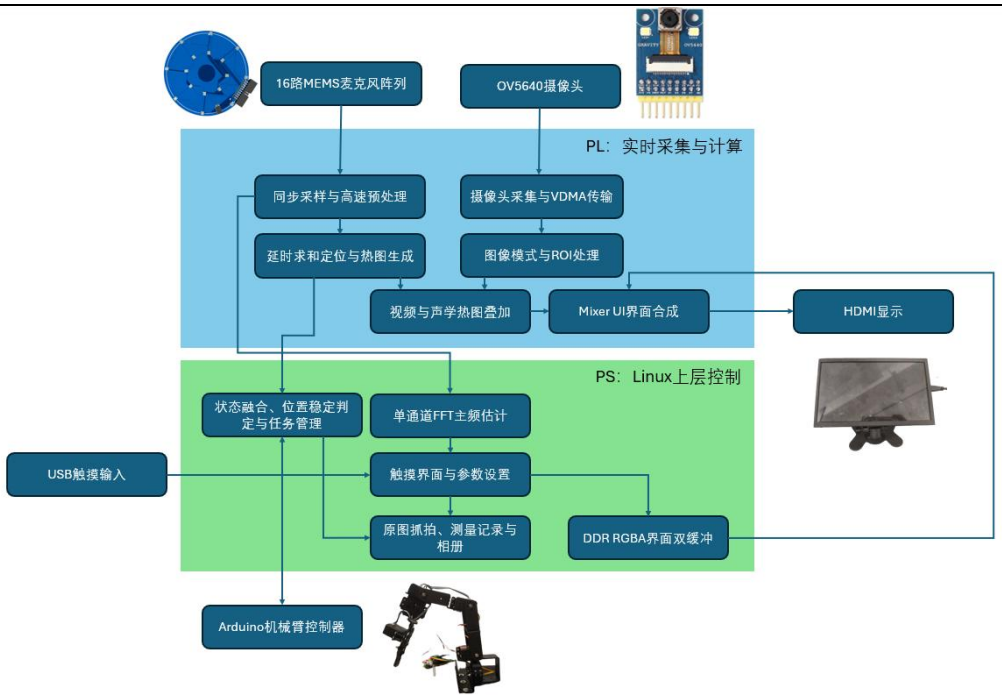


图 2-1　系统总体功能框图

感知前端采用内环半径 2.5 cm、外环半径 5 cm 的 16 路 ICS-43434 麦克风阵列，并配合 OV5640 摄像头获取声音与图像信息。麦克风数据进入 PL 后完成同步接收、高通预处理及延时求和计算，生成 64×36 声学网格及相应热图；摄像头数据经过采集、缓存和图像处理后，与声学热图叠加。

Linux 将界面渲染至 DDR 中的 RGBA 双缓冲区，由 PL 侧 Mixer 完成界面与主视频的最终合成，再通过 HDMI 输出。USB 触摸事件由 Linux 接收，用于完成模式切换、参数调整、抓拍浏览和机械臂操作。

2.1.3 软硬件划分与接口关系

系统按照任务的吞吐量、时序要求及交互特点进行 PS/PL 划分：持续运行、并行度高且时序敏感的采样与像素计算放在 PL；状态管理、显示策略、文件存储和通信任务放在 Linux。

表 2-1 软硬件划分

部分	主要职责	划分依据
PL 声学链路	多通道采样、高通预处理、延时求和及热图生成	多路数据需并行处理，并保持确定的计算时序
PL 视频链路	图像采集、VDMA 传输、图像模式、ROI 及图层合成	视频数据量大，适合流式流水线处理
Linux 控制服务	UIO 访问、静音状态融合、色阶调整及位置稳定判定	侧重状态与策略，便于修改和调试
Linux 交互与存储	触摸界面、参数设置、抓拍及相册	涉及交互逻辑、文件系统和动态界面
Linux 频率与通信服务	单通道 FFT、机械臂任务发送及回包解析	便于独立运行、更新和扩展协议
机械臂独立控制器	动作执行与结果回传	由独立主控完成机械动作控制

PS 通过 AXI 寄存器接口配置 PL 中的图像模式、ROI、色阶及图层参数，并读取声学定位和运行状态。Linux 通过 UIO 身份识别打开对应设备，避免依赖固定的设备编号。视频帧缓冲与 UI 缓冲区位于 DDR 中，通过 VDMA 及 Mixer 等模块与 PL 视频通路协同工作。

2.1.4 系统工作流程

系统启动后，Linux 依次完成摄像头、视频传输、显示时序及相关处理模块的初始化，随后启动声学控制、频率估计、触摸界面和机械臂通信服务。

运行过程中，PL 持续处理声音与图像数据；Linux 读取定位和能量状态，结合静音门限、滞回及位置稳定判定，更新热点显示、坐标和操作目标。声学检测开启且目标位置有效时，可进一步驱动声源跟随 ROI 或生成机械臂区域任务。

抓拍时，系统读取摄像头原图，并将本次请求的定位、频率及运行状态保存为配套测量记录。机械臂联动时，手动模式依据最新有效实时坐标确定区域，自动模式依据稳定目标和驻留条件触发；Linux 发送带任务号的命令，独立控制器执行后回传结果，由任务服务维护状态并更新界面。

2.2 硬件系统介绍

2.2.1 硬件整体介绍

本系统以 ZYNQ-7020 开发板为核心，连接 16 路 MEMS 麦克风阵列、OV5640 摄像头、HDMI 触摸显示屏及外部机械臂控制器，构成声音采集、图像采集、实时计算、显示交互与执行联动的硬件平台。

ZYNQ 的 PL 部分承担麦克风数据接收、声学预处理、延时求和定位、视频处理及图层合成；PS 部分运行 Linux，通过 AXI 接口配置硬件参数、读取运行状态，并管理触摸、存储和机械臂通信。板载 DDR 作为视频帧、声学图像及 UI 缓冲区的存储空间，SD 卡用于系统启动和抓拍记录保存。

表 2-2 硬件模块及功能

硬件模块	主要组成	功能
主控模块	ZYNQ-7020、板载 DDR、SD 卡	实时计算、系统控制与数据存储
声学采集模块	16 个 ICS-43434 数字麦克风	多通道声音同步采集
视觉采集模块	OV5640 摄像头	实时图像采集
显示交互模块	HDMI 显示屏、USB 触摸接口	图像显示、参数设置与操作输入
执行接口模块	PS UART1、独立 Arduino 控制器	机械臂命令发送与结果回传
供电与散热模块	5V 电源、散热片及风扇	为各模块供电并辅助散热

2.2.2 机械布局与阵列结构

麦克风阵列采用双同心圆 PCB 布局，内环半径为 2.5 cm，外环半径为 5 cm，每环布置 8 个麦克风，共形成 16 路采集通道。

表 2-3 麦克风硬件安排

阵列部分	半径	麦克风数量	数据通道
内环	2.5 cm	8	data0～data7
外环	5 cm	8	data8～data15

各通道按照实物编号建立坐标对应关系，并据此生成定位计算使用的延迟参数。由于内、外环编号起点不同，算法中的通道坐标分别按实际接线配置。

机械布局以阵列平面及摄像头相对位置作为声视坐标映射的安装基准。测试时保持两者相对位置固定，并使麦克风入声孔周围留有空间，减少安装面遮挡。主控板、显示屏、电源与散热模块采用模块化连接，便于调试和后续整机装配。

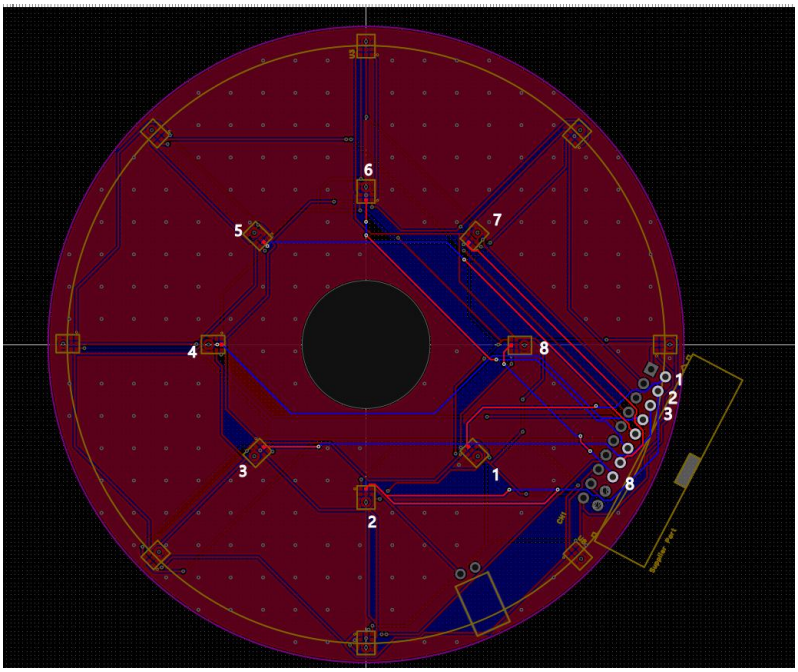


图 2-2 麦克风阵列 PCB

2.2.3 电路与接口模块

（1）麦克风采集接口

ICS-43434 采用数字串行输出，16 个麦克风共用 SCK 和 WS 时钟，各自通过独立 SD 线向 FPGA 传输数据。麦克风采用 3.3V 供电，LR 引脚接地，接收逻辑按左声道时隙读取 24 位有符号样本。[4]

当前设计由 30 MHz 时钟分频产生 3 MHz SCK 及 46.875 kHz WS，满足每个采样周期 64 个 SCK 的时序关系：

$$f_s = \frac{f_{\mathrm{SCK}}}{64} = \frac{3\,000\,000}{64} = 46\,875\ \mathrm{Hz}$$

sensors 模块并行接收 16 路串行数据，输出 384 位样本总线及样本有效信号，供后续高通和定位模块使用。

（2）摄像头接口

OV5640 通过 8 位 DVP 接口输出图像数据，主要信号包括：

表 2-4 摄像头主要信号

信号	方向	作用
DATA[7:0]	摄像头→FPGA	像素字节
PCLK	摄像头→FPGA	像素采样时钟
HREF	摄像头→FPGA	行有效指示
VSYNC	摄像头→FPGA	帧同步指示
SCCB SCL/SDA	控制交互	摄像头寄存器配置与读取

摄像头配置为 1280×720、RGB565 输出。PL 采集模块在 PCLK 时钟域完成字节接收与像素拼接，将 RGB565 扩展为 RGB888，再转换为 AXI4-Stream 视频流。SCCB 由 PS 侧 GPIO 控制，用于初始化、参数配置和诊断读取。[5]

实际连接中，PCLK 与 GND 配对连接，两端分别接至摄像头和开发板地，以改善摄像头连接线的信号传输条件。

（3）显示与触摸接口

视频通过 PL 中的显示输出链路转换为 TMDS 信号，经 HDMI 接口送至显示屏。触摸数据通过 USB Host 接口进入 PS，由 Linux 输入子系统接收，并交给 UI 程序处理。

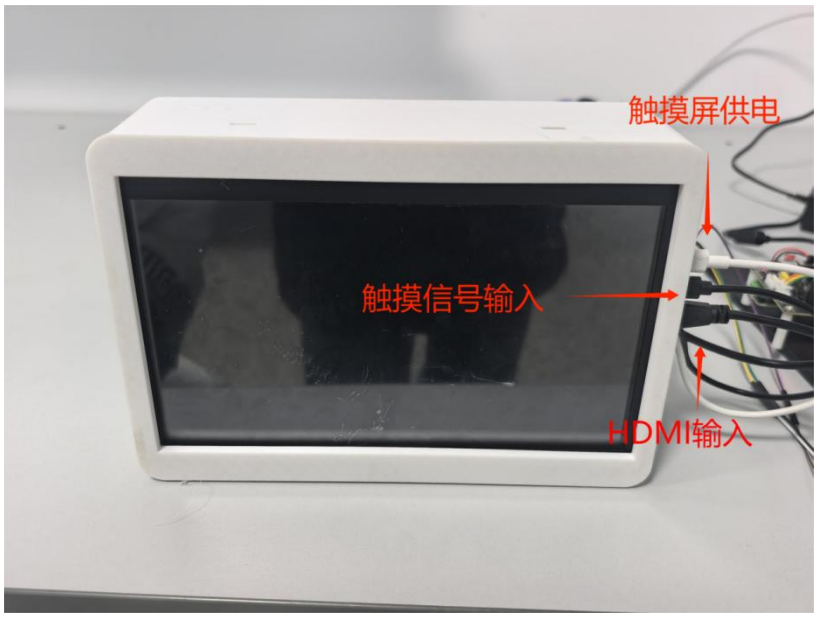


图 2-3 显示与触摸接口

显示与触摸分别承担输出和输入功能：PL 负责最终视频合成，Linux 根据触

摸事件更新界面及控制参数。

（4）机械臂通信接口

机械臂通信采用 PS UART1，经 EMIO 引出，串口配置为 9600 bit/s、8 位数据位、无校验、1 位停止位。

ZYNQ 发送带任务号的夹取、跟踪或测试命令，Arduino 独立控制器负责动作执行并回传结果。通信接口将上层检测任务与底层动作控制连接起来。

（5）供电与散热

系统采用 5V 供电，麦克风由开发板提供 3.3V 电源，各信号模块保持公共参考地。便携供电使用 HJ-28UPS 电池电源模块，开发板配置散热片和风扇。

片上 XADC 周期采集芯片温度，Linux 界面显示当前温度并进行分级提示，为现场观察设备热状态和调整散热提供依据。

2.2.4 PL 逻辑与数据通路

（1）声学处理链路

声学处理按以下流程运行：

16 路采样→200 Hz 高通→样本缓存与延迟读取→延时求和及网格能量更新→伪彩色映射→声学视频生成。

sensors 完成串并转换；acoustic_hpf_200 对各通道执行高通预处理，抑制直流和低频背景分量；buffer_ctrl、calculate 及 vram_ctrl 协同完成样本管理和定位计算，得到 64×36 声学网格。[2,6]

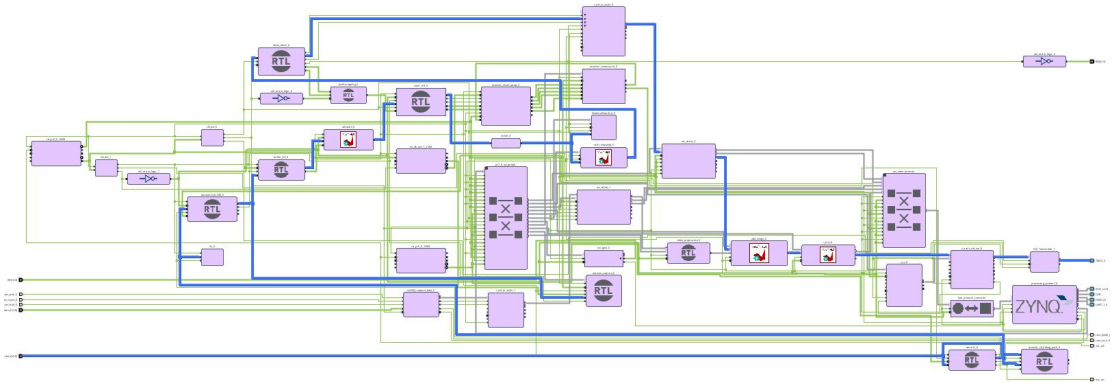


图 2-5 声学处理链路

网格能量经像素映射及 color_mapping 转换为伪彩色热图，进入声学视频缓

存链路。64×36 网格提供候选方向的离散表达，并通过阵列几何与画面坐标映射实现声源位置显示。

（2）视频处理与合成链路

当前工程使用两路 VDMA，分别缓存摄像头图像和声学图像：

表 2-5 视频流处理模块

模块	对应数据	后续连接
axi_vdma_1	摄像头图像	video_preprocess
axi_vdma_0	声学图像	add_image
video_preprocess	摄像头图像模式与 ROI 处理	add_image
add_image	摄像头与声学图像叠加	v_mix
v_mix	主视频与 RGBA 界面合成	视频输出模块

video_preprocess 支持原图、灰度、二值化、亮度偏移、反相和移位增益六种模式。处理区域可限定为矩形 ROI，框外保持原图；Linux 通过更新 ROI 坐标实现手动框选或声源跟随。配置经提交后在帧首生效，避免同一帧中途切换参数。

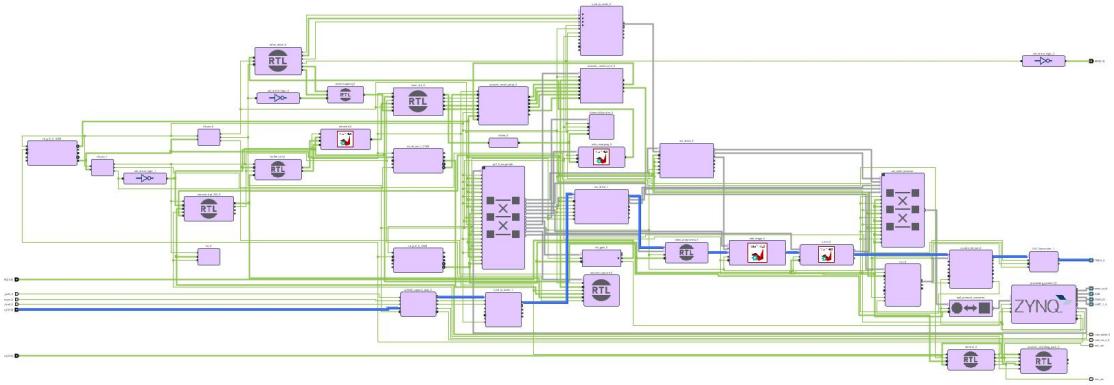


图 2-6 视频处理链路

add_image 完成声学图像与摄像头画面的叠加。v_mix 从 DDR 读取 Linux 生成的 RGBA 界面，将其合成至主视频。VTC 提供显示时序，视频输出模块完成流格式转换和 TMDS 发送。[7,8]

（3）单通道样本采集支路

acoustic_capture 从高通输出旁路选取一路麦克风数据，连续采集 2048 点后冻结至片上 BRAM，供 PS 读取。该支路保留原声学数据通路，不暂停定位计算。

FFT 主频估计由 Linux 完成，PL 负责样本选择、采集、缓存及完成状态传

递。

2.2.5 控制接口、时钟与跨域设计

PS 经 M_AXI_GP0 上的控制互联访问 PL 外设的 AXI4-Lite 寄存器，配置图像模式、ROI 及图层参数。PL 侧 VDMA 与 Mixer 作为 AXI 主设备，经 PS 的 S_AXI_HP0 访问 DDR 中的视频和界面缓冲区；Linux 在预留 DDR 区域绘制 RGBA 界面，再由 Mixer 读取合成。主要自定义接口如下：[3]

表 2-6 自定义模块

模块	基地址	作用
acoustic_camera_ctrl	0x44A20000	声学控制与状态读取
acoustic_capture	0x40030000	单通道样本采集
video_preprocess	0x40040000	图像模式、ROI 及区域统计
v_mix	0x44A30000	UI 图层与合成参数控制

当前工程的主要时钟配置如下：

表 2-7 主要时钟配置

时钟	配置频率	主要用途
PS FCLK0	100 MHz	部分 AXI 控制外设及视频采集通路
PS FCLK1	150 MHz	视频读出、图像处理、Mixer 及 DDR 访问
声学运算时钟	210 MHz	声学计算、缓存及高通处理
麦克风分频基准	30 MHz	生成 SCK 和 WS
显示像素时钟	74.25 MHz	视频显示及声学图像生成
显示发送时钟	371.25 MHz	TMDS 串行发送

摄像头PCLK作为独立输入时钟，视频通过Video In模块转换至AXI4-Stream通路。声学结果通过 acoustic_result_adapter 的保持与请求/应答机制传入 AXI 控制域；单通道采集模块通过跨域握手传递采集请求和完成状态。

约束中对同步器入口、多位数据保持路径及总线偏斜分别处理。调试阶段使用 ILA 观测麦克风样本、有效信号和处理状态，为接收时序及通道一致性验证提供依据。

2.3 软件系统介绍

2.3.1 软件整体介绍

系统软件运行于 ZYNQ 的 PS 端，以 PetaLinux 为基础，采用“硬件访问—后台服务—交互应用”的分层结构。PL 持续完成声学与视频计算，Linux 负责硬件初始化、状态融合、显示策略、人机交互、文件存储及机械臂任务管理。

软件主要采用 C 语言实现，各功能通过独立服务或后台任务运行。进程之间使用 Unix 域 Socket、运行时状态文件和管道交换数据，使频率采集、文件读取及串口通信与 UI 绘制分开执行。系统核心功能均在板端运行，PC 用于工程开发、构建和调试。

软件模块及主要输入输出如下表所示。

表 2-8 软件模块

软件模块	主要程序	关键输入	主要输出
视频初始化	video_pipeline_init	摄像头参数、硬件配置	视频链路配置、启动状态
声学显示控制	acoustic_display_control	网格能量、噪声基线、开关状态	色阶、叠加透明度、声音状态
定位状态服务	pl_control_daemon	定位寄存器、声音状态	实时坐标、稳定坐标、位置有效性
频率估计	acoustic_frequency 及采集服务	单通道样本块	主峰频率、有效性及采样信息
交互界面	video_status_panel	触摸、测量与设备状态	RGBA 界面、操作请求
图像控制	video_preprocess_control	模式、参数、ROI	PL 配置事务及生效结果
抓拍与相册	camera_snapshot、camera_album	原图缓冲、测量信息、历史文件	BMP、JSON、缩略图和浏览数据
机械臂控制	robot_control_daemon	操作请求、定位及串口回包	控制命令、任务与连接状态
UI 运行监测	video_ui_watchdog	UI 进程、视频节拍及 DMA 状态	监测状态、条件恢复请求

各模块通过状态接口连接，UI 主要承担显示和操作入口，具体计算、采集和通信由对应后台模块完成，具体可见下图。

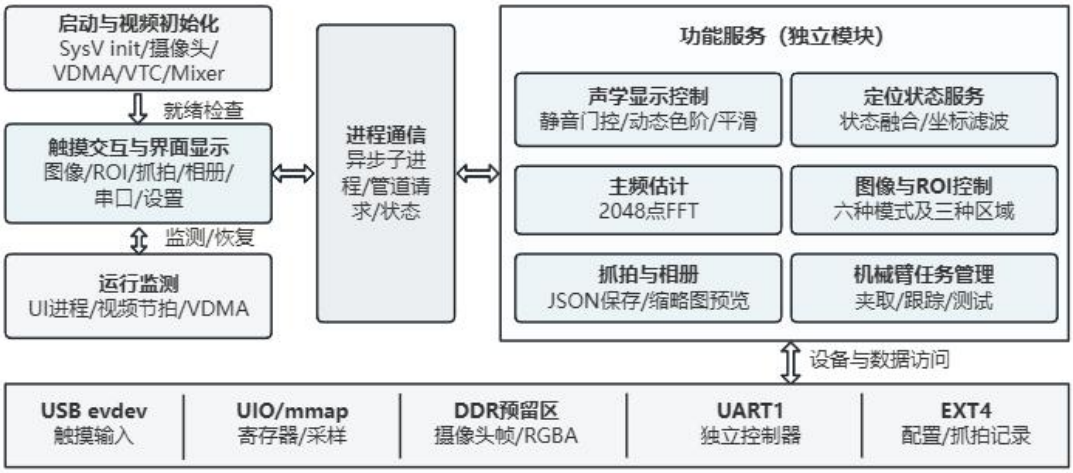


图 2-7 Linux 软件模块与数据流框图

2.3.2 软件各模块介绍

（1）系统启动与视频初始化

系统从 SD 卡启动，依次完成 FSBL、U-Boot 及 Linux 加载，并挂载 EXT4 根文件系统。各应用通过 SysV 启动脚本运行，根据硬件和视频状态进行有界等待。

video_pipeline_init 负责摄像头 SCCB 配置、HLS 默认参数、两路 VDMA、VTC 和 Mixer 的初始化。主要流程为：

硬件身份核对 → Mixer 主视频层准备 → HLS 参数及 VDMA 配置 → 启动 S2MM → 摄像头配置与开流 → 检查采集帧 → 启动 MM2S 并配置 VTC → 发布运行状态。

程序将启动阶段和返回结果写入/run/video-pipeline.state，详细信息写入日志，便于区分摄像头配置、采集传输及显示输出故障。

（2）UIO 身份识别与硬件访问

软件通过/sys/class/uio 枚举设备，联合检查设备名称、物理基地址和映射大小，确定对应的/dev/uioN，再通过 mmap 访问寄存器。[10]

对于自定义模块，进一步核对硬件标识、协议版本和功能能力。该方式使软件能够适应设备探测顺序变化，并在接口不匹配时拒绝写入。涉及同一硬件资源的程序通过互斥锁协调访问。

（3）声学显示控制与逻辑开关

acoustic_display_control 读取声场最大值、最小值及相关状态，根据已保存的背景噪声基线完成静音门控。程序结合开启/关闭双门限、连续观测和时间平滑，减少孤立尖峰引起的显示变化。

声音有效时，程序依据声场能量调整颜色阈值和叠加透明度，并限制色阶变化速度；静音或数据不可用时撤去热图。

声学开关由 pl_control_daemon 接收请求，显示控制服务执行透明度调整并发布回读确认。关闭后清空定位历史，停止位置输出及相关控制联动；麦克风采样、PL 计算和单通道频率采集继续运行。

（4）定位状态融合与稳定判定

pl_control_daemon 周期性读取定位寄存器，结合声学显示服务发布的声音状态、开关确认和数据新鲜度，生成统一的定位状态。

程序对重复读取的一致性进行检查，将硬件坐标转换为 64×36 网格坐标，再利用滑动窗口、中值滤波、空间聚集程度和驻留时间判断位置是否稳定。主要状态包括静音、不可用、获取中、移动和稳定。

服务同时提供实时坐标、滤波坐标、稳定状态、观测年龄及读取质量。静音、关闭或来源失效时清空历史，避免继续使用旧目标。UI 及机械臂服务通过 /run/pl-control.sock 查询这些结果。

（5）单通道主频估计

频率服务读取 PL 采集模块冻结的 2048 点单通道样本，依次进行去均值、Hann 加窗、FFT、谱峰搜索及峰值插值，得到当前输入的主峰频率候选。

采集服务将结果原子更新至/run/acoustic-frequency.state。UI 结合谱峰有效性、削顶状态、数据年龄和声音状态决定是否显示读数。

该频率来自高通后的单个麦克风通道，用于提供当前声音的频谱参考。

（6）触摸界面与双缓冲显示

video_status_panel 采用轻量 RGBA 绘制方式，结合离线生成的中文字形及数字字库，完成顶部状态栏、测量侧栏、控制底栏、参数抽屉和机械臂窗口绘制，目标刷新频率为 5 Hz。

触摸输入通过 Linux 输入事件接口读取，根据设备能力识别触摸屏，并将坐

标映射至 1280×720 画布。程序处理短按、拖动和滑动操作，对断线、多指及异常事件取消当前手势。

界面使用 DDR 双缓冲：CPU 绘制后台缓冲，写入完成后提交 Mixer 更新，等待完成事件后再复用缓冲区。抓拍、相册读取等操作以后台任务执行，主循环持续处理触摸和状态更新。

（7）图像模式与 ROI 控制

图像控制支持原图、灰度、二值化、亮度偏移、反相和移位增益六种互斥模式。用户调整参数后，软件完成范围检查，将模式、阈值及 ROI 等参数作为一次完整事务提交，并检查硬件生效结果。

处理区域支持全屏、手动框选和声源跟随。跟随模式使用稳定网格坐标，其对应的显示像素中心为：

$$x_c = 20c + 10, \quad y_c = 20r + 10$$

其中，c、r 分别为网格列和行。软件根据用户设置的宽、高生成矩形，并进行边界限制、移动死区及步长控制。声源状态失效后撤去跟随处理，避免旧位置持续驱动 ROI。

（8）抓拍、测量记录与相册

camera_snapshot 首先检查摄像头 VDMA 配置、运行状态、帧推进及存储空间，满足条件后读取摄像头原图缓冲。复制过程中检查写槽变化、错误状态和数据一致性，无法满足保护条件时返回失败。

每次成功抓拍保存 camera.bmp 及配套 measurement.json，记录定位、能量、单通道频率、温度和软件时间信息。原图与测量按同次请求关联保存，关联时间写入记录。

文件先写入临时目录，经同步后发布为正式记录。系统同时生成 160×90 缩略图供相册列表使用；详情页按需读取原图和测量数据。相册文件处理由后台进程完成，UI 负责分页、选择和结果展示。

（9）机械臂任务管理

robot_control_daemon 独占 UART1，采用 9600 bit/s、8N1 配置。接收程序根据消息类型确定帧长度，校验帧头、帧尾、任务号及字段，支持分片接收、连续回包和失步重对齐，并区分 UART 线上时序与 Linux 批量交付间隔。

机械臂页面提供三类功能：

表 2-9 机械臂功能简介

功能	软件处理方式
夹取	手动模式使用最新有效位置，确认发送时重新确定区域；自动模式要求稳定目标驻留，并设置触发锁定、静音重装和冷却条件
跟踪	将稳定位置映射至已标定区域，自动更新目标并维护通信状态
测试	按用户确认执行指定次数或连续测试，并提供停止后续动作入口

任务发送前保存任务号和状态，接收回包后更新执行进度，并通过请求去重和事务互斥防止重复下发。系统采用上电默认不使能自动动作的安全策略，操作人员确认工作区后通过界面开启自动模式。回包超时记为结果未知，由任务状态机结束本地等待，避免自动重发造成重复动作。

（10）运行监测与关机管理

系统在/run 发布高频运行状态，配置、照片和关键任务记录保存在 EXT4 中。UI 读取 XADC 温度，显示温度等级及设备运行状态；设置页提供关机确认入口，调用 Linux 正常关机流程。

2.4 大模型协作与技能包提炼

2.4.1 协作目标与任务划分

机械臂功能扩展时，系统出现短执行回包可正常解析、长标定回包持续超时的现象。开发团队结合接线条件、串口协议和接收计数，借助大模型分析数据交付过程、设计最小复现样例并提出修改方案；团队完成现场操作与结果确认，通过离线回放和板端配置查询验证修复效果。

协作采用“现象与证据整理、候选解释、最小对照、板端验证、经验提炼”的流程。团队分别核对字节接收、完整组帧和业务响应，使每项结论对应到可检查的数据。关键问答摘录及复用入口见附录 A。

2.4.2 问题定位与假设修正

调试初期，UART1 采用 9600 bit/s、8N1 配置，接收程序把连续两批数据交付间隔超过 50 ms 作为清除未完成帧的条件。五次配置查询累计接收 650 字节，

却未形成有效响应，并记录五次半包过期，表明故障集中在接收后的组帧环节。

结合 RX FIFO 的 56 字节水位，团队进一步核查线上传输与用户态交付的时间关系。8N1 格式每字节占 10 位，56 字节在 9600 bit/s 下传输约 58.33 ms；驱动可能将连续响应分批交付给应用。实测 130 字节响应按 56、56、18 字节到达，相邻交付间隔为 59 ms 和 22 ms，首个间隔已超过原程序的 50 ms 判据。

据此，优化方案将主机半包交付阈值独立设为 200 ms，同时保留协议长度、消息类型、任务号匹配及完整请求截止时间。判断依据由“交付间隔是否超过单一阈值”调整为“帧接收进度与事务期限是否分别满足要求”，从而适应驱动分批交付。

2.4.3 对照验证与结果

采用同一 130 字节合成响应及 56/56/18 字节交付轨迹，对修正前后的 C 解析器进行对照。原实现得到 0 个完整帧，优化后得到 1 个完整帧；真正的主机空闲和请求总超时仍按设定生效。

修复后的板端配置查询共接收 163 字节，经 4 批交付解析出 2 个有效响应，未出现半包过期、拒绝或忽略。该结果验证了优化方案对长回包接收问题的修复效果。

最终技能包包含 30 项离线测试，覆盖分片、连续帧、噪声、截断、异常帧尾、主机空闲、错误任务号和请求总超时等情况。C 参考实现与参数化 Python 模型的行为对照全部通过，验证结果随技能包提供。

2.4.4 技能提炼与适用边界

团队将本案例提炼为 linux-uart-receive 技能包，整理诊断流程、最小轨迹、回放脚本、参考实现及验证样例。使用者可替换波特率、消息长度与交付轨迹，分析同类串口分批接收问题。200 ms 阈值对应本案例的交付条件，迁移至其他控制器时应结合缓冲水位和调度间隔重新设置；不同帧格式则需适配解码规则。

2.5 工程复现与运行方法

2.5.1 最小复现准备

最小复现直接使用配套预编译文件，无需安装 Vivado、Vitis 或 PetaLinux。若使用正点原子 ZYNQ-7020 最小系统板可按以下图示连接开发板、阵列、OV5640、机械臂。HDMI 触摸屏需要连接供电、HDMI 输入以及触摸数据输入。

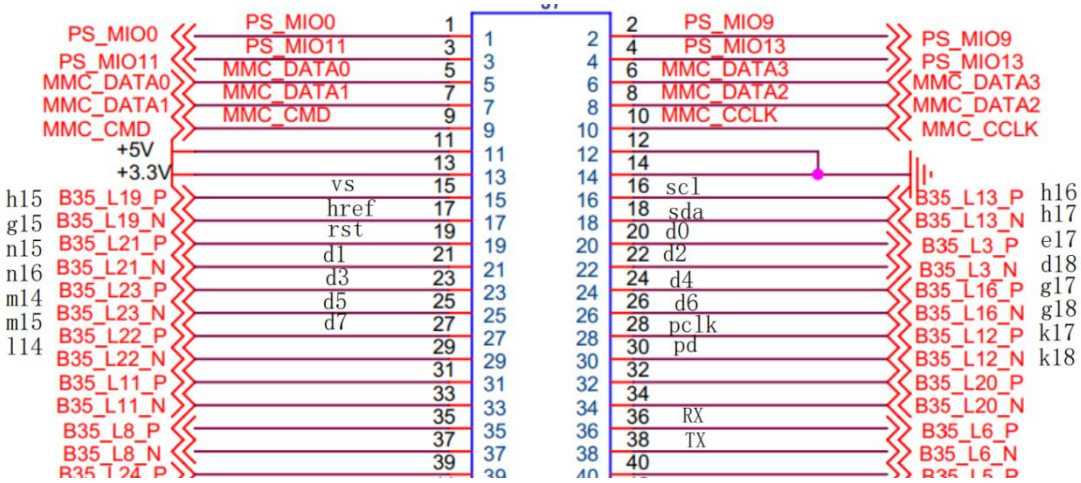


图 2-8 摄像头与串口连接

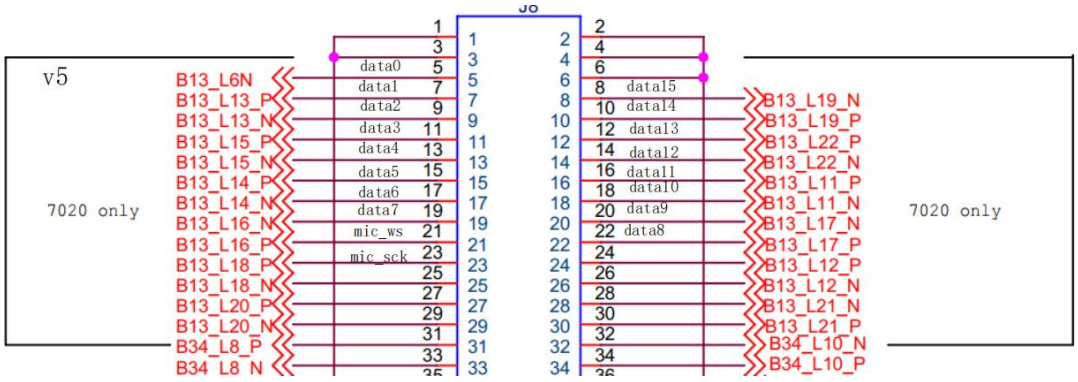


图 2-9 麦克风阵列连接

所需文件见表 2-10。rootfs.ext4 须包含最终版程序、配置和自启动脚本；完整运行镜像在源码包和 github 仓库提供。

表 2-10 最小复现所需文件

文件	写入位置与用途
BOOT.BIN	SD 第一分区 FAT32 根目录；装载 PL 和启动程序
image.ub	SD 第一分区 FAT32 根目录；Linux 内核等启动内容
boot.scr	SD 第一分区 FAT32 根目录；U-Boot 启动脚本
system.dtb	SD 第一分区 FAT32 根目录；配套设备树
rootfs.ext4	镜像写入 SD 第二分区；根文件系统和应用服务

2.5.2 SD 卡写入

在 Linux 主机或虚拟机中核对、备份 SD 卡，准备 FAT32 启动分区和 EXT4 根分区。复制四个启动文件到第一分区，卸载第二分区后写入 rootfs.ext4。以下占位路径须替换为已核对的 SD 第二分区，不能指向整卡或电脑系统盘。

```
lsblk -o NAME,MODEL,SIZE,FSTYPE,MOUNTPOINTS
ROOT_PART='/dev/disk/by-id/<SD 设备标识>-part2'
test -b "$ROOT_PART" || exit 1
test -f rootfs.ext4 || exit 1
sudo dd if=rootfs.ext4 of="$ROOT_PART" bs=4M status=progress conv=fsync
sudo e2fsck -f "$ROOT_PART"
sudo resize2fs "$ROOT_PART"
sync
```

文件系统检查若有未修复错误，应停止排查，不继续扩容。写入完成后安全卸载两个分区，将 SD 卡插回开发板。rootfs.ext4 必须写入第二分区，不能只作为普通文件放入启动分区。[9]

2.5.3 上电启动

选择 SD 启动模式，连接 HDMI、触摸 USB 和 UART0 调试串口后上电。串口使用 115200 bit/s、8N1。系统自动装载 PL 并启动 Linux、视频、声学及 UI 服务，机械臂自动功能默认关闭。登录后检查根目录为 SD 第二分区的可写 EXT4，视频状态正常。

```
findmnt /
cat /run/video-pipeline.state
```

2.5.4 首次校准

首次在当前环境使用时，停止测试声源并保持安静，以 root 权限执行约 5 秒背景校准，保存本机噪声门限后启动显示策略。

```
/etc/init.d/acoustic-display-control stop
/usr/bin/acoustic_display_control --calibrate
/etc/init.d/acoustic-display-control start
```

2.5.5 功能验收

确认摄像头连续更新、触摸正常；在阵列前约 30 cm 播放固定 2 kHz 测试音，检查热点、坐标和频率，移动声源后热点跟随，停声后消退。抓拍一次并从相册打开，确认图像与测量记录保存；退出时通过设置页正常关机，完成后断电。

源码构建及机械臂任务测试见附录 A.1。详细制卡、首次登录、校准与故障排查方法以及源码重新构建方法见源码包/reproduce 下的相关文档。上述文档同时在项目 GitHub 仓库提供。

第三部分　完成情况及性能参数

3.1 整体介绍

整机系统由便携式触摸终端、背板双环麦克风阵列与外部六自由度机械臂组成，实物联调场景如图 3-1 所示。

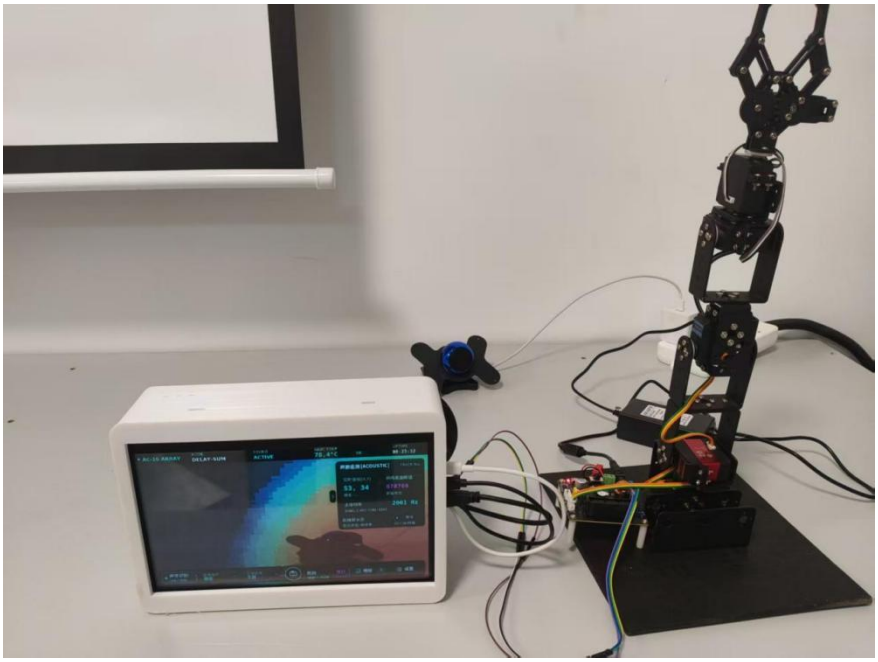


图 3-1 作品整体展示

3.2 工程成果

3.2.1 机械成果

系统外壳采用 3D 打印定制，预留显示屏窗口、散热孔位及背部阵列同轴安装孔，如图 3-2 所示。

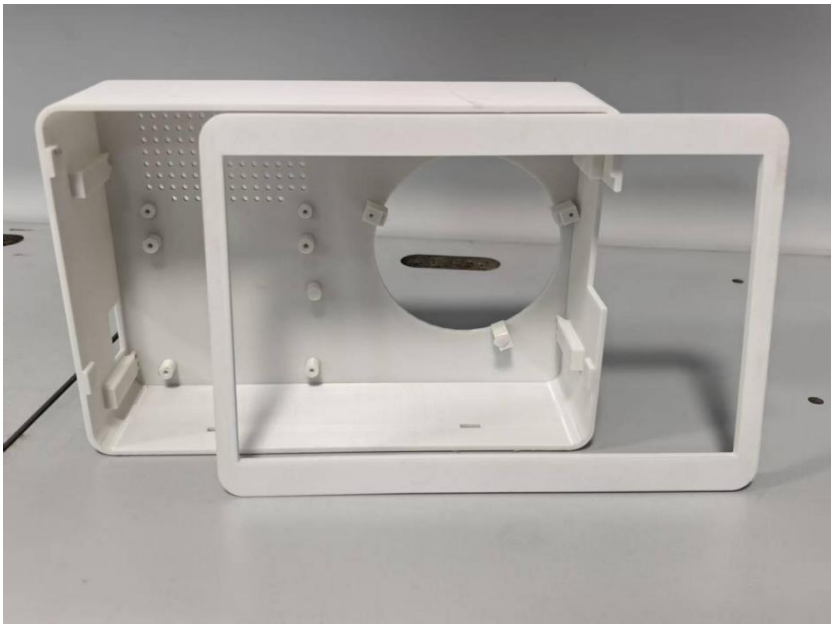


图 3-2 作品机械外壳

3.2.2 电路成果

核心硬件连接涵盖 16 路双同心圆 MEMS 麦克风阵列 PCB（中心同轴嵌入 OV5640 摄像头）、ZYNQ-7020 核心板及便携电源模块，如图 3-3 所示。

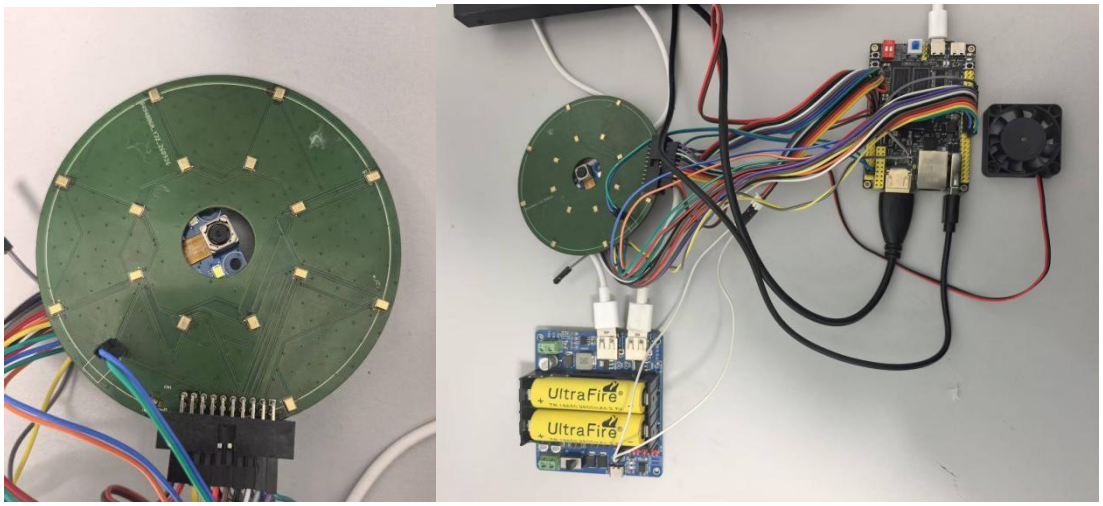


图 3-3 电路成果

3.2.3 软件成果

软件界面集成了声场伪彩热图叠加、顶部运行状态栏、右侧声源遥测面板与底部多功能控制栏，如图 3-4 所示。

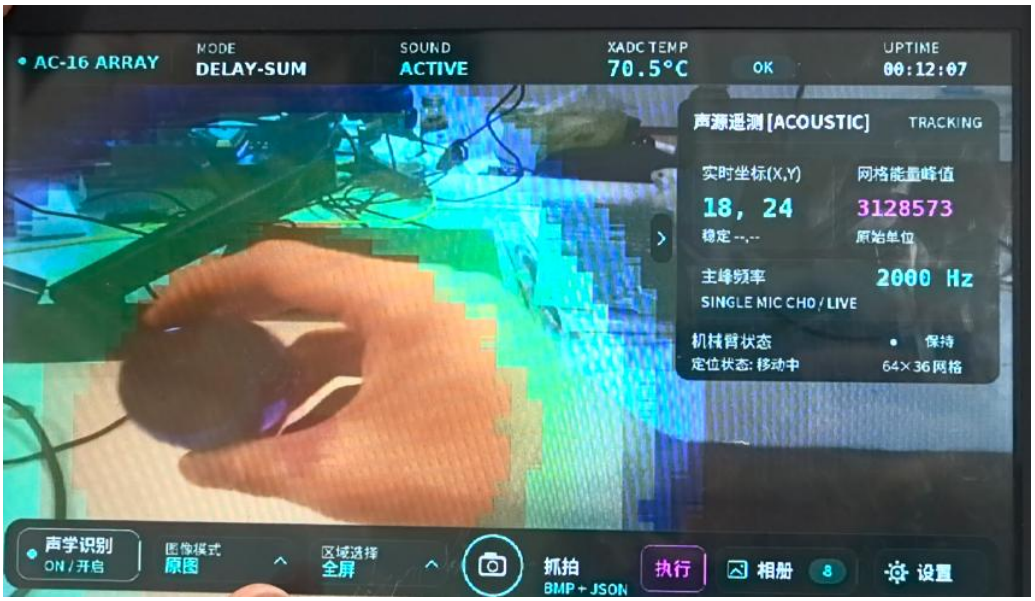


图 3-4 软件交互界面

3.3 特性成果

样机围绕声源定位、视觉辅助处理、触摸交互、测量记录和区域执行接口开展功能验证，以下给出测试条件、对照数据及实现结果。

3.3.1 声源可视化定位与主峰频率显示

样机以 16 路双环阵列、46.875 kHz 采样和 64×36 能量网格完成声源成像，并与 1280×720 摄像头画面叠加。固定 2 kHz 或 3 kHz 单音在约 30 cm 处形成局部热点；声源移动时热点与实时坐标随之更新，保持位置后显示稳定坐标，停止播放后叠加消退。触摸关闭和重新开启声学识别，可分别撤去与恢复热点及有效定位。

主峰频率使用 CH0 高通后的 2048 点数据在 Linux 侧估计，作为现场声音的频谱参考。FFT 频点间隔约 22.89 Hz，经 Hann 加窗与峰值插值后，五组合成单音的离线验证结果见表 3-1。

表 3-1 五组合成单音的离线频率估计结果

合成输入频率/Hz	估计频率/Hz	绝对偏差/Hz
800.0	799.912	0.088
997.3	997.077	0.223
2000.0	2000.307	0.307
3000.0	3000.132	0.132
3500.0	3499.849	0.151

五组离线单音的最大绝对偏差为 0.307 Hz，直流、静音、弱声、双音及削顶等测试也通过。板端以手机播放单音，将界面读数与设定频率对照，观察偏差约为±2 Hz；两类结果分别用于验证离线估计算法和现场频率读数。

现场粗测中，系统在约 5 m 处可获得声源位置引导。约 800 Hz 时热点较宽，可辨认左、中、右方向；约 3.5 kHz 时对桌面和人体反射更敏感，调整安装、使阵列脱离桌面后可重复观察到单热点。测试时保持阵列、声源和周边条件稳定，有助于获得一致的定位显示。

3.3.2 背景抑制与热点稳定性改善

针对静音背景上色和低音量下的热点波动，系统加入 200 Hz 高通，并结合静音滞回、动态色阶及时间平滑。在相同环境、显示配置与噪声门限下，依次测试高通开启、旁路和恢复开启三种状态；每段等待 1 秒后，以 250 ms 间隔读取 24 次状态，结果见表 3-2。

表 3-2 静音场景下的高通开关对照

阶段	背景峰值统计中位数	最小值～最大值
A：高通开启	643	396～1130
B：高通旁路	38593.5	13996～302194
C：恢复高通	690	458～1577

高通开启两阶段的峰值统计中位数平均为 666.5，旁路时为 38593.5，后者约为前者的 57.9 倍；恢复高通后背景统计回到相近水平。该数值反映 PL 能量网格背景峰值的变化，表明高通预处理有效抑制了本测试环境中的背景分量。

加入门控与平滑后，静音场景中的背景闪色消失，声源移动时热点继续跟随，停止播放后正常消退。低音量及较远距离场景下的热点显示也得到改善。

3.3.3 图像处理与触摸交互成果

视频链路实现了原图、灰度、二值化、亮度偏移、反相和移位增益六种模式，并支持全屏、手动矩形及声源跟随三类处理区域。板端测试中，二值化和亮度偏移滑块可拖动、保留数值并在应用后改变图像效果；区域设置用于限定处理范围，框外保留原图。声学热图和 UI 位于图像处理之后，图像模式切换不改变二者的配色。

触摸界面形成了顶部状态栏、可收起半透明测量侧栏、底部功能栏及功能抽屉。顶部显示阵列、算法、声音状态、XADC 温度和运行时间；侧栏显示实时/稳定坐标、能量、主峰频率及机械臂状态。模式切换、区域选择、侧栏收放、参数滑块、抓拍相册和关机入口均经过板端触摸确认，摄像头画面可在交互过程中持续更新。



图 3-5 主体页面

UI 以 RGBA 双缓冲和独立图层合成，目标更新周期为 200 ms。摄像头与声学流水线持续运行，菜单绘制、触摸响应和测量状态更新由 Linux 完成。

3.3.4 原图抓拍与测量记录成果

抓拍功能保存 1280×720 摄像头原图 BMP，以及同次软件请求读取的测量 JSON；同时生成 160×90 缩略图，供相册快速浏览。记录可保留定位、能量、频率、温度及有效性状态，详情页以 640×360 预览和普通文本展示测量内容，便于回看及后续分析。

在 30 cm 摄像头接线、PCLK/GND 绞合及降频配置下，连续多次读取摄像头 S2MM 错误掩码均为 0，写缓冲保持轮换。命令行与触摸入口各完成一次抓拍，结果见表 3-3。

表 3-3 两次板端抓拍与归档结果

抓拍入口	单次任务耗时	保存结果
命令行请求	779 ms	BMP、测量记录及缩略图保存成功
触摸抓拍按钮	886 ms	保存成功，相册计数增加

两次抓拍与归档耗时约 0.8～0.9 秒，均成功保存并增加相册记录。随后可通

过触摸浏览列表、照片和详细数据，摄像头继续更新。图像读取后立即获取测量状态，以同次软件事务及时间戳关联保存，便于回看时对应现场测量信息。

3.3.5 机械臂区域任务与串口通信成果

系统建立了 Linux 到独立机械臂控制器的 UART 任务接口，界面提供夹取、跟踪和测试三个功能页。手动夹取使用最新有效实时坐标确定左、中、右区域，确认窗口随目标更新；自动夹取和跟踪保留稳定位置、驻留和任务互斥条件。任务号匹配、重复请求去重、持久化和超时处理用于避免重复下发及误认完成。

左、中、右三个料位的独立动作测试均完成了命令发送、执行中回包和流程完成回包，现场观察到正常动作、放置与返回，验证了区域任务下发、动作执行及状态回传链路。界面和任务服务负责把有效坐标转换为操作区域，将声源引导与独立控制器的动作能力连接起来。

针对长回包，接收程序区分 UART 线上传输与 Linux 分批交付间隔。优化后的板端配置查询共接收 163 字节并形成 2 个有效响应，长回包超时得到修复。相关诊断与验证方法提炼为 UART 技能包，可用于同类控制器接口的接收问题排查。

3.3.6 FPGA 资源与时序成果

系统在 XC7Z020-CLG400-2 上完成布局布线，声学、视频、UI 合成及调试测量模块的资源占用如图 3-6 所示。

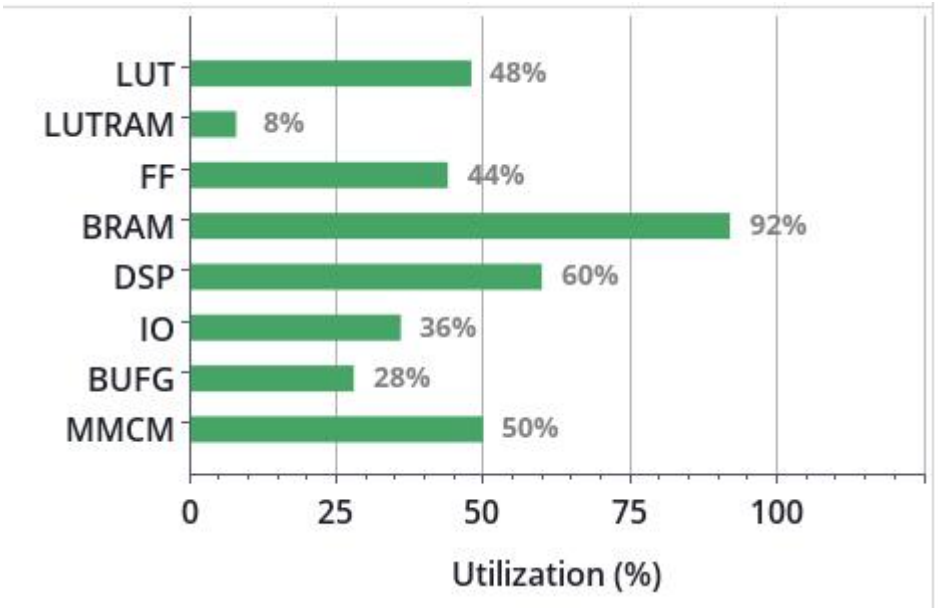


图 3-6　PL 端资源占用

实现后的建立时间裕量 WNS 为+0.136 ns，保持时间裕量 WHS 为+0.036 ns，TNS、THS、布线错误及 DRC Error 均为 0，满足设计已定义的时序约束。BRAM 占用 92.14%，DSP 占用 60.45%；功能扩展优先采用缓存复用和 DDR 协同，兼顾计算吞吐与片上存储开销。

第四部分　总结

4.1 可扩展之处

后续围绕定位标定、复检闭环与整机可靠性扩展。开展阵列幅相及相机视场标定，按频率、声强、距离和反射条件量化定位误差，引入分频段处理及空间频谱关联。结合现有区域动作能力，完善阵列、相机与机械臂坐标标定及夹持反馈，加入执行后复测和结果比较。工程上优化 BRAM 复用、DDR 调度与显示同步，完善硬件同帧记录、供电隔离、功耗及续航评估，并扩展记录导出和远程维护。

4.2 心得体会

本项目从开源声学相机出发，逐步完成双环阵列适配、Linux 移植、触摸界面及机械臂联动。研发过程中，我们对软硬件协同有了更具体的认识：算法、采样、时序、驱动和物理连接都会影响最终效果，模块验证与整机验证需要分别开

展。

Linux 移植中，裸机可正常工作的硬件曾出现黑屏、四分屏和新旧帧交替。团队对照裸机寄存器，逐项检查 GPIO 映射与时钟、SCCB 应答、VDMA 启动顺序和 VTC 参数，逐步恢复视频。这使我们形成了按数据流分层排查的习惯：错误清除、传输触发和启动条件都应依据接口语义处理。

声学调试中，早期存在静音闪色、音量变化导致热点面积剧烈波动等问题。我们先核对 16 路通道与阵列几何映射，再利用 ILA 检查原始 SD 及接收样本，逐步加入 200 Hz 高通、静音门限、滞回和色阶平滑，改善了显示稳定性。测试也表明，频率、背景噪声、桌面反射和人体站位均可能影响热点，因此比较算法前后效果时，需要固定声源位置、音量和安装条件。

制作细节同样影响系统可靠性。摄像头接线由 21 cm 加长至 30 cm 后，暗部出现绿噪点；缩短接线后恢复，PCLK 与 GND 配对绞合也带来明显改善。这使我们更加重视高速信号的回流路径和线缆布局。供电方面，触摸 USB 还可能形成额外供电路径，整机上电、掉电和 USB 连接顺序都应纳入验证。

UI 与抓拍联调表明，寄存器和软件计数正常并不等于视频输出正常。为此，系统补充状态观测及条件式 UI 恢复，并保留现场日志分析显示失步。结合较高的 BRAM 占用，新增功能前需评估缓存复用与 DDR 访问，将资源预算纳入设计。

大模型协作提高了代码整理和测试脚本编写效率，建议仍须经源码、仿真及板端反馈核验。UART 案例最终提炼为含失败样例、修复对照和验证脚本的技能包。后续将完善版本与构建记录，以统一条件下的量化测试支撑性能结论。

第五部分　参考文献

[1] wzwyz. ZYNQ-based-acoustic-camera[EB/OL]. [2026-10-31].

[2] VAN VEEN B D, BUCKLEY K M. Beamforming: a versatile approach to spatial filtering[J]. IEEE ASSP Magazine, 1988, 5(2): 4-24. DOI: 10.1109/53.665.

[3] AMD. Zynq 7000 SoC Technical Reference Manual: UG585[R/OL]. Version 1.15. 2026-02-06[2026-10-31].

[4] INVENSENSE INC. ICS-43434: Multi-Mode Microphone with I2S Digital Output: DS-000069[R/OL]. Revision 1.2. 2016-08-29[2026-10-31].

[5] OMNIVISION TECHNOLOGIES INC. OV5640: color CMOS QSXGA (5 megapixel) image sensor with OmniBSI technology[R/OL]. Version 2.03. 2011-05-26[2026-10-31].

[6] AMD. Vitis High-Level Synthesis User Guide: UG1399[R/OL]. Version 2026.1. 2026-09-11[2026-10-31].

[7] AMD. AXI Video Direct Memory Access v6.3 Product Guide: PG020[R/OL]. 2022-06-08[2026-10-31].

[8] AMD. Video Mixer LogiCORE IP Product Guide: PG243[R/OL]. Version 6.1. 2026-07-31[2026-10-31].

[9] AMD. PetaLinux Tools Documentation: Reference Guide: UG1144[R/OL]. Version 2026.1. 2026-06-23[2026-10-31].

[10] KOCH H J. The Userspace I/O HOWTO[EB/OL]. (2006-12-11)[2026-10-31].

附录 A 复现与协作资料

A.1 工程入口与构建运行

工程源码、构建脚本、板端说明和技能包统一随作品提交。公开入口为 https://github.com/rk-3588/zynq-acoustic-visual-inspection，复现时以交付清单记录的源码及二进制版本为准。下列命令在源码根目录执行，路径按复现主机填写。

（1）安装 Vivado 2026.1、PetaLinux 2026.1 及 Zynq-7000 器件支持，在 Linux 主机核验源码、编译主机程序并运行回归。

```
python3 build/audit_release.py
python3 build/build_ps.py
python3 build/test_software.py
```

（2）在 Vivado 2026.1 环境创建独立工程并构建硬件。Windows 使用短输出路径；生成的位流、XSA 和实现报告位于工程输出目录。

```
vivado -mode batch -source build/create_project.tcl -tclargs D:/ac16_build
vivado -mode batch -source build/build_hardware.tcl -tclargs
D:/ac16_build/ac16_final.xpr
```

（3）加载 PetaLinux 2026.1 环境，新建 Zynq 工程，暂存配方和配置后导入本次 XSA，设置本地账号与 EXT4 根文件系统，再构建镜像。暂存脚本的替换选项会备份目标工程原配置。

```
petalinux-create project --template zynq --name ac16_linux
SOURCE_ROOT=/path/to/ac16_source
PETA_PROJECT=/path/to/ac16_linux
python3 "$SOURCE_ROOT/build/stage_petalinux.py" "$PETA_PROJECT" --replace
cd "$PETA_PROJECT"
petalinux-config --get-hw-description /path/to/xsa_directory
petalinux-config -c rootfs
petalinux-build
```

（4）将同次构建的 FSBL、位流、U-Boot 和设备树放入打包目录，以如下顺序编写 boot.bif，再生成 BOOT.BIN。

```
all: {
[bootloader] zynq_fsbl.elf
system.bit
u-boot.elf
[load=0x00100000] system.dtb
}
bootgen -arch zynq -image boot.bif -o BOOT.BIN -w on
```

（5）准备 FAT32 启动分区和 EXT4 根分区，将四个启动文件复制到第一分区。备份、核对并卸载 SD 第二分区后写入 rootfs.ext4，再运行文件系统检查和扩容。目标分区名称由 lsblk 核对，写入对象为第二分区。

```
lsblk -o NAME,MODEL,SIZE,FSTYPE,UUID,MOUNTPOINTS
SD_ROOT='/dev/disk/by-id/CONFIRMED_SD_DEVICE-part2'
test -b "$SD_ROOT" || exit 1
sudo dd if=images/linux/rootfs.ext4 of="$SD_ROOT" bs=4M status=progress conv=fsync
sudo e2fsck -f "$SD_ROOT"
sudo resize2fs "$SD_ROOT"
```

（6）完成同步和安全卸载后，以 SD 启动板卡。串口控制台使用 UART0、115200/8N1；Linux 启动参数如下。

```
console=ttyPS0,115200 earlycon root=/dev/mmcblk0p2 rw rootwait uio_pdrv_genirq.of_id=generic-uio
```

（7）登录后检查根挂载、UIO 及视频状态，随后在安静环境下完成约 5 秒背景校准。

```
findmnt /
cat /run/video-pipeline.state
/usr/bin/video_display_diag --samples 3 --interval-ms 1000
/etc/init.d/acoustic-display-control stop
/usr/bin/acoustic_display_control --calibrate
/etc/init.d/acoustic-display-control start
```

（8）依次验收摄像头连续更新、触摸操作、固定单音热点及停声消退、六种图像模式与 ROI、抓拍相册。机械臂先核对 UART1 的 9600/8N1、电平与区域映射，再在工作区确认后测试任务及回包。退出时通过设置页正常关机。

A.2 大模型协作摘录与技能复用

以下以机械臂串口长回包超时为例，依据实际协作记录与验证日志精简整理问题、模型回答及验证结果。

提问：Linux 串口已经收到数据，为什么长标定回包仍然超时，而短回包能够正常解析？

模型回答（精简）：应区分串口线上的字节间隔与 Linux 用户态 read()的交付间隔。在 9600 bit/s、8N1 下，56 字节传输约 58.3 ms；日志中 130 字节响应按 56、56、18 字节交付，批次间隔为 59 ms 和 22 ms。原程序以 50 ms 判定半包过期，导致正常长帧被误清除。

将主机半包交付阈值单独设为 200 ms，并保留固定帧长、TYPE 与任务号匹配及总请求期限。先用相同分批轨迹回放，再用只读配置查询复核；本案例不改波特率、FPGA 或控制器固件。200 ms 为本案例参数，其他平台应依据交付日志重新确定。

验证结果：同一 130 字节合成轨迹回放，修复前 0 个完整帧、1 次半包过期，修复后 1 帧、0 次过期；板端两项只读配置查询得到 2 帧、0 次过期。

技能包以 skill/README.md 为入口，说明适用场景、使用步骤和失效条件。复用时替换交付轨迹及协议参数，先完成离线回放，再接入自身控制器。

A.3 指标说明

表 1-1 中的约 5 m 为现场定位引导粗测；主频显示偏差来自手机单音设定值与屏幕读数的观察对照。表 3-1 使用已知采样率下的合成序列，给出五组离线估计偏差。表 3-3 记录两次独立抓拍任务耗时，图像与测量以同次事务及时间戳关联保存。